1. HTTP - 웹 기초

웹 브라우저, 서버, 웹 애플리케이션은 모두 **HTTP(HyperText Transfer Protocol)** 를 통해 서로 대화한다. HTTP는 현대 인터넷의 공용어다.

1.1 HTTP : 인터넷의 멀티미디어 배달부

HTTP는 전 세계 웹 서버에 있는 대량의 정보를 빠르고, 간편하고, 정확하게 사람들의 컴퓨터에 설치된 웹 브라우저로 옮겨준다. **HTTP는 신뢰성 있는 데이터 전송 프로토콜(TCP)을 사용**하기 때문에 데이터가 전송 중에 손상되거나 꼬이지 않음을 보장한다.

이는 개발자에게도 이롭다. 데이터가 전송 중에 파괴되거나, 중복되거나, 왜곡되는 것을 걱정하지 않고 애플리케이션 고유의 기능 구현에만 집중할 수 있기 때문이다.

1.2 웹 클라이언트와 서버

웹 서버는 HTTP 프로토콜로 의사소통하기 때문에 보통 **HTTP 서버**라고 부른다. 웹 서버는 인터넷의 데이터를 저장하고, HTTP 클라이언트가 요청한 데이터를 제공한다.

클라이언트는 서버에게 HTTP 요청을 보내고, 서버는 요청된 데이터를 HTTP 응답으로 돌려준다. 가장 흔한 HTTP 클라이언트는 웹 브라우저다. 주소창에 URL을 입력하면 브라우저가 서버에 요청을 보내고, 받은 응답을 화면에 그려준다.

> HTTP 클라이언트와 HTTP 서버는 월드 와이드 웹의 기본 요소다.

1.3 리소스

웹 서버는 **웹 리소스(웹 콘텐츠의 원천)** 를 관리하고 제공한다. 가장 단순한 웹 리소스는 웹 서버 파일 시스템에 있는 정적 파일이다.

```
예) 정적 파일 → 텍스트 파일, HTML 파일, 워드 파일, JPEG 파일 등
```

그러나 리소스가 반드시 정적 파일이어야 하는 것은 아니다. 요청에 따라 콘텐츠를 생성하는 프로그램도 리소스가 될 수 있다. 로그인한 사용자에 따라 다른 화면을 보여주거나, 주식 시세를 실시간으로 보여주는 프로그램이 그 예다. 즉, **어떤 종류의 콘텐츠 소스도 리소스가 될 수 있다.**

1.3.1 미디어 타입

HTTP는 인터넷의 수천 가지 데이터 타입을 다루기 때문에, 웹에서 전송되는 객체마다 **MIME(Multipurpose Internet Mail Extensions, 다목적 인터넷 메일 확장)** 타입이라는 데이터 포맷 라벨을 붙인다. MIME은 원래 서로 다른 전자메일 시스템 사이에서 메시지를 주고받을 때 생기는 문제를 해결하려고 설계되었는데, HTTP가 이를 가져와 사용하고 있다.

웹 서버는 모든 HTTP 객체 데이터에 MIME 타입을 붙인다. 웹 브라우저는 서버로부터 객체를 돌려받으면 MIME 타입을 보고 자신이 다룰 수 있는 객체인지 확인한다. 이 라벨은 응답의 `Content-Type` 헤더에 담긴다.

MIME 타입은 `/`로 구분된 **주 타입(primary object type)** 과 **부 타입(specific subtype)** 으로 이루어진 문자열 라벨이다.

콘텐츠	MIME 타입
HTML 문서	`text/html`
일반 ASCII 텍스트	`text/plain`
JPEG 이미지	`image/jpeg`
GIF 이미지	`image/gif`

콘텐츠	MIME 타입
JSON 데이터	`application/json`
파일 업로드(폼)	`multipart/form-data`

> 브라우저는 확장자가 아니라 **MIME 타입**으로 콘텐츠를 판단한다. 그래서 서버가 `.html` 파일을 `text/plain`으로 보내면, 브라우저는 페이지를 렌더링하지 않고 HTML 소스를 글자 그대로 보여준다.

1.3.2 URI : Uniform Resource Identifier

웹 서버 리소스는 각자 이름을 갖고 있어서, 클라이언트는 원하는 리소스를 지목할 수 있다. 서버 리소스의 이름을 **통합 자원 식별자(Uniform Resource Identifier), 줄여서 URI**라고 부른다. URI는 인터넷의 우편물 주소 같은 것으로, 정보 리소스를 고유하게 식별하고 위치를 지정할 수 있다.

```
https://www.example.com/special/something.gif
```

URI에는 URL과 URN, 두 종류가 있다.

URL : Uniform Resource Locator

- 통합 자원 지시자(URL)는 리소스 식별자의 가장 흔한 형태다. URL은 특정 서버에 있는 한 리소스의 구체적인 위치를 서술한다. 리소스가 정확히 어디에 있고 어떻게 접근할 수 있는지 분명히 알려준다.
- 대부분의 URL은 세 부분으로 이루어진 표준 포맷을 따른다.
  - **첫 번째 부분** : **스킴(scheme)** 이라고 부르며, 리소스에 접근하는 데 사용하는 프로토콜을 서술한다.
  - **두 번째 부분** : 서버의 인터넷 주소(호스트)를 제공한다.
  - **세 번째 부분** : 웹 서버의 리소스(경로)를 가리킨다.

```
https://www.example.com/special/something.gif
└─┬─┘   └──────┬──────┘└──────────┬──────────┘
  스킴        호스트              경로
```

URN : Uniform Resource Name

- 유니폼 리소스 이름(URN)은 콘텐츠를 이루는 한 리소스에 대해, **리소스의 위치에 영향받지 않는** 유일무이한 이름 역할을 한다.
- 위치와 독립적이기 때문에 리소스를 여기저기 옮기더라도 문제없이 동작한다.

```
urn:isbn:9788966261208   → ISBN으로 특정 책을 가리킨다 (어디에 있는지는 말하지 않는다)
urn:ietf:rfc:2141        → RFC 2141 문서를 가리킨다
```

> URN은 아직 널리 쓰이지 않아서, 현재 웹에서 사용하는 URI는 사실상 대부분 URL이다. 그래서 실무에서는 URI와 URL을 거의 같은 의미로 섞어 쓴다.

1.4 트랜잭션

> HTTP 트랜잭션은 **요청 명령**과 **응답 결과**로 구성된다. 이 상호작용은 **HTTP 메시지**라고 불리는 정형화된 데이터 덩어리를 이용해 이루어진다.

1.4.1 메소드

HTTP는 **HTTP 메소드**라고 불리는 여러 종류의 요청 명령을 지원한다. 모든 HTTP 요청 메시지는 메소드를 하나씩 갖는다. **메소드는 서버에게 어떤 동작을 취해야 하는지 알려준다.**

메소드	설명
`GET`	서버에서 클라이언트로 지정한 리소스를 보내라
`POST`	클라이언트 데이터를 서버 게이트웨이 애플리케이션으로 보내라
`PUT`	**클라이언트에서 서버로 보낸 데이터를 지정한 이름의 리소스로 저장하라**
`DELETE`	지정한 리소스를 서버에서 삭제하라
`HEAD`	지정한 리소스에 대한 응답에서 HTTP 헤더 부분만 보내라

PUT 메소드

> 개발자 면접을 준비하거나 CS 공부를 하다 보면 PUT, PATCH 메소드를 접하게 된다. 상당히 많은 자료가 PUT과 PATCH를 '수정'에 관한 메소드라고 설명한다. 하지만 이 책은 '저장'이라는 동사로 PUT을 간략히 서술했다. 당장은 깊게 다루지 않고 넘어가지만, PUT과 PATCH의 동작을 **멱등성**과 **안전성**에 연관 지어 너무 깊지는 않더라도 알고 가면 남들과는 다른 답변을 할 수 있지 않을까 하는 생각이 들었다.

PUT이 '저장'인 이유는, PUT이 리소스의 **일부를 수정**하는 것이 아니라 보낸 데이터로 리소스 **전체를 대체**하기 때문이다. 리소스가 없으면 새로 만들고, 있으면 통째로 덮어쓴다. 그래서 같은 PUT 요청을 여러 번 보내도 결과가 같다. 반면 PATCH는 리소스의 **일부만** 변경한다.

- **안전(safe)** : 요청이 서버의 리소스를 변경하지 않는다.
- **멱등(idempotent)** : 같은 요청을 한 번 보내든 여러 번 보내든 서버의 최종 상태가 같다.

메소드	안전	멱등
`GET`, `HEAD`	O	O
`PUT`	X	O

메소드	안전	멱등
`DELETE`	X	O
`POST`	X	X
`PATCH`	X	보장되지 않음

PATCH는 `{"op": "increment", "path": "/count"}`처럼 "1 증가시켜라" 같은 요청도 표현할 수 있다. 이런 요청은 보낼 때마다 결과가 달라지므로 멱등하지 않다. 멱등성이 중요한 이유는 **재시도** 때문이다. 네트워크 오류로 응답을 못 받았을 때, 멱등한 요청은 안심하고 다시 보낼 수 있다.

1.4.2 상태 코드

모든 HTTP 응답 메시지는 상태 코드와 함께 반환된다. 상태 코드는 클라이언트에게 요청이 성공했는지, 아니면 추가 조치가 필요한지 알려주는 **세 자리 숫자**다.

상태 코드	의미
`200`	좋다. 문서가 바르게 반환되었다.
`302`	다시 보내라. 다른 곳에 가서 리소스를 가져가라.
`404`	없음. 리소스를 찾을 수 없다.

상태 코드는 첫 번째 자리로 범주가 나뉜다. 처음 보는 코드라도 첫 자리만 보면 대략적인 의미를 알 수 있다.

범위	범주	예시
`1xx`	정보	`100 Continue`
`2xx`	성공	`200 OK`, `201 Created`
`3xx`	리다이렉션	`301 Moved Permanently`, `302 Found`
`4xx`	클라이언트 오류	`400 Bad Request`, `404 Not Found`

범위	범주	예시
5xx	서버 오류	`500 Internal Server Error`, `503 Service Unavailable`

HTTP는 상태 코드와 함께 `OK`, `Not Found` 같은 **사유 구절(reason phrase)** 도 보낸다. 사유 구절은 사람이 읽기 위한 설명일 뿐이고, 클라이언트는 숫자 코드로 동작을 결정한다.

1.5 메시지

HTTP 메시지는 단순한 줄 단위의 문자열이다. 이진 형식이 아닌 일반 텍스트이기 때문에 사람이 읽고 쓰기 쉽다.

- **웹 클라이언트에서 웹 서버로 보낸 HTTP 메시지를 요청 메시지라고 부른다.**
- **서버에서 클라이언트로 가는 메시지를 응답 메시지라고 부른다.**

HTTP 메시지는 다음 세 부분으로 이루어진다.

```
a : 요청 메시지
GET /test/hi-there.txt HTTP/1.1      ---> 시작줄
Host: www.example.com                ---> 헤더
Accept: text/*                       ---> 헤더
Accept-Language: en, fr              ---> 헤더

b : 응답 메시지
HTTP/1.1 200 OK                      ---> 시작줄
Content-Type: text/plain             ---> 헤더
Content-Length: 12                   ---> 헤더
                                     ---> 빈 줄
Hello World!                         ---> 본문
```

시작줄

- 메시지의 첫 줄은 시작줄이다. 요청이라면 **무엇을 해야 하는지**, 응답이라면 **무슨 일이 일어났는지** 나타낸다.
- 요청의 시작줄은 `메소드 경로 버전`, 응답의 시작줄은 `버전 상태코드 사유구절` 형식이다.

헤더

- 시작줄 다음에는 0개 이상의 헤더 필드가 이어진다.
- 각 헤더 필드는 쉬운 구문 분석을 위해 `:`로 구분된 하나의 이름과 하나의 값으로 구성된다. (이름과 `:` 사이에는 공백을 두지 않는다)
- 헤더 필드를 추가하려면 그저 한 줄을 더하기만 하면 된다.
- HTTP/1.1 요청에서는 `Host` 헤더가 필수다. 하나의 IP에서 여러 도메인을 운영하는 서버가 어느 사이트에 대한 요청인지 구분하기 위해서다.

본문

- 헤더가 끝나면 **빈 줄**이 오고, 그다음에 어떤 종류의 데이터든 들어갈 수 있는 메시지 본문이 필요에 따라 올 수 있다.
- `Content-Length`는 본문의 바이트 수다. `Hello World!`는 12바이트이므로 `Content-Length: 12`가 된다.

> 텍스트 기반 메시지는 HTTP/1.x의 특징이다. HTTP/2부터는 같은 의미의 메시지를 성능을 위해 **이진 프레임**으로 나눠 전송한다. 메소드, 헤더, 상태 코드의 의미는 그대로다.

1.6 TCP 커넥션

> TCP : Transmission Control Protocol (전송 제어 프로토콜)

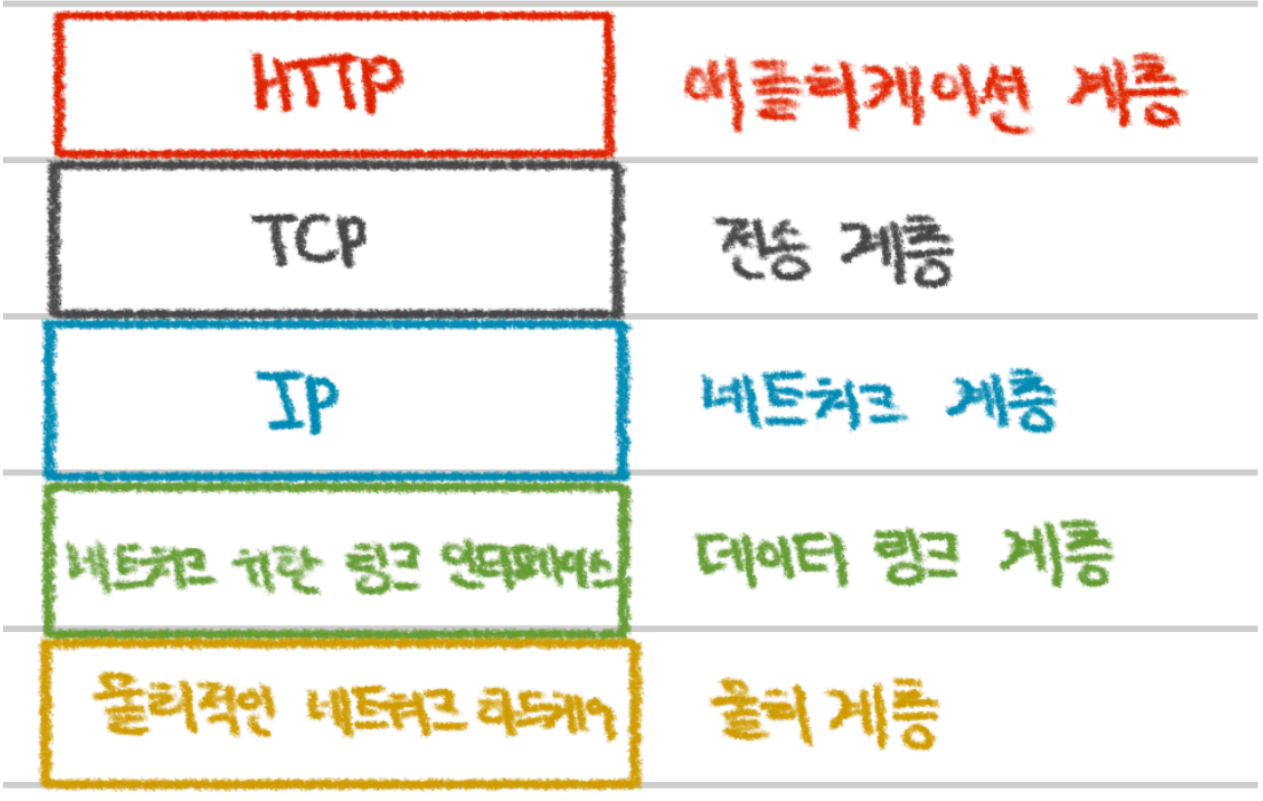
1.6.1 TCP/IP

HTTP는 **애플리케이션 계층 프로토콜**이다. HTTP는 네트워크 통신의 핵심적인 세부사항은 신경 쓰지 않고, 대중적이고 신뢰성 있는 인터넷 전송 프로토콜인 **TCP/IP**에게 맡긴다. TCP는 다음을 제공한다.

- **오류 없는 데이터 전송**
- **순서에 맞는 전달** : 데이터는 언제나 보낸 순서대로 도착한다.
- **조각나지 않는 데이터 스트림** : 언제든 어떤 크기로든 보낼 수 있다.

인터넷 자체가 전 세계의 컴퓨터와 네트워크 장치들 사이에서 대중적으로 사용되는 TCP/IP에 기초하고 있다. TCP/IP는 **TCP와 IP가 층을 이루는 패킷 교환 네트워크 프로토콜의 집합**이다. IP는 패킷을 목적지 주소까지 전달하고, TCP는 그 위에서 유실·중복·순서를 바로잡아 신뢰성을 만든다. TCP/IP는 네트워크와 하드웨어의 특성을 숨기고, 어떤 종류의 컴퓨터나 네트워크든 서로 신뢰성 있게 의사소통할 수 있게 해준다.

일단 TCP 커넥션이 맺어지면, 클라이언트와 서버 사이에 교환되는 메시지가 없어지거나, 손상되거나, 순서가 뒤바뀌어 수신되는 일은 없다.



1.6.2 접속, IP 주소, 포트 번호

서버에 메시지를 전송하려면 먼저 **IP 주소와 포트 번호**를 사용해 클라이언트와 서버 사이에 TCP/IP 커넥션을 맺어야 한다. TCP에서는 서버 컴퓨터의 IP 주소와, 그 서버에서 실행 중인 프로그램이 사용하는 포트 번호가 필요하다. HTTP 서버의 IP 주소와 포트 번호는 URL을 이용해 알 수 있다.

```
http://207.200.83.20:80/index.html
http://www.netscape.com/index.html
```

첫 번째 URL은 IP 주소와 포트 번호를 직접 담고 있다. 두 번째 URL에는 IP 주소 대신 호스트 명이 있고 포트 번호가 없다. 호스트 명은 **DNS(Domain Name Service)** 를 통해 IP 주소로 변환하고, 포트 번호가 없으면 스킴의 **기본 포트**를 사용한다. (`http`는 80, `https`는 443)

웹 브라우저가 URL을 받아 문서를 보여주기까지의 과정은 다음과 같다.

1. 웹 브라우저는 서버의 URL에서 호스트 명을 추출한다.
2. 웹 브라우저는 서버의 호스트 명을 IP 주소로 변환한다. (DNS)
3. 웹 브라우저는 URL에서 포트 번호를 추출한다.
4. 웹 브라우저는 웹 서버와 TCP 커넥션을 맺는다.
5. 웹 브라우저는 서버에 HTTP 요청을 보낸다.
6. 서버는 웹 브라우저에 HTTP 응답을 돌려준다.
7. 커넥션이 닫히면 웹 브라우저는 문서를 보여준다.

> 7번처럼 요청마다 커넥션을 닫는 방식은 초기 HTTP의 동작이다. 커넥션을 맺는 비용이 크기 때문에, HTTP/1.1부터는 기본적으로

> 커넥션을 열어둔 채 여러 요청을 재사용하는 **지속 커넥션 (persistent connection)** 을 사용한다.

1.7 웹의 구성요소

구성요소	설명
`Proxy`	클라이언트와 서버 사이에 위치한 HTTP 중개자
`Cache`	많이 찾는 웹 페이지를 클라이언트 가까이에 보관하는 HTTP 창고
`Gateway`	다른 애플리케이션과 연결된 특별한 웹 서버
`Tunnel`	단순히 HTTP 통신을 전달하기만 하는 특별한 프록시
`Agent`	자동화된 HTTP 요청을 만드는 준지능적 웹 클라이언트

1.7.1 Proxy

프록시는 웹 보안, 애플리케이션 통합, 성능 최적화를 위한 중요한 구성요소다.

프록시는 클라이언트와 서버 사이에 위치해 클라이언트의 모든 HTTP 요청을 받아 서버에 전달한다. 즉, **사용자를 대신해서 서버에 접근**한다. 주로 보안을 위해 사용되며, 모든 웹 트래픽 흐름 속에서 신뢰할 만한 중개자 역할을 한다. 요청과 응답을 필터링하기도 한다. 예를 들어 학교나 회사에서 특정 사이트 접속을 막거나, 다운로드 파일의 바이러스를 검사하는 데 쓰인다.

> 클라이언트 쪽에서 사용자를 대신하는 프록시를 **포워드 프록시**, 서버 앞에서 서버를 대신해 요청을 받는 프록시를 **리버스 프록시**라고 부른다. Nginx를 웹 서버 앞에 두는 구성이 대표적인 리버스 프록시다.

1.7.2 Cache

웹 캐시와 프록시 캐시는 자신을 거쳐 가는 문서 중 자주 찾는 것의 사본을 저장해두는 특별한 종류의 HTTP 프록시 서버다. 다음에 클라이언트가 같은 문서를 요청하면, 캐시가 갖고 있는 사본을 받을 수 있다. 멀리 있는 서버까지 가지 않아도 되므로 응답이 빠르고, 서버의 부하와 네트워크 사용량도 줄어든다.

HTTP는 캐시를 효율적으로 동작하게 하고, 캐시된 콘텐츠를 최신 버전으로 유지하면서, 동시에 프라이버시도 보호하기 위한 많은 기능을 제공한다. (`Cache-Control`, `ETag` 같은 헤더가 그 예다)

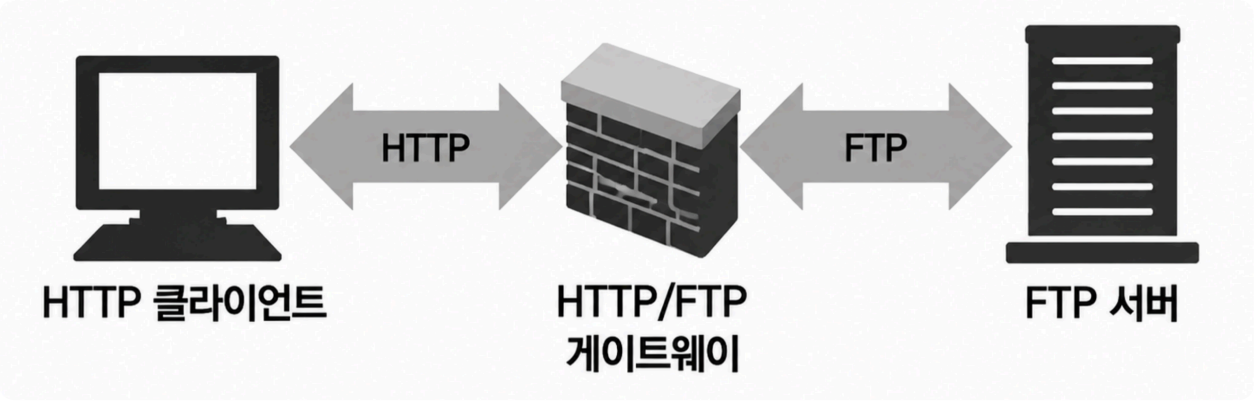
```
Client1, Client2, ... <--> Proxy Cache <--> Server
```

1.7.3 Gateway

> 클라이언트와 실제 서버 사이에서 요청을 받아, 다른 서버나 시스템으로 전달한다.

게이트웨이는 다른 서버들의 중개자로 동작하는 특별한 서버다. 주로 **HTTP 트래픽을 다른 프로토콜로 변환**하기 위해 사용된다. 게이트웨이는 언제나 스스로 리소스를 갖고 있는 진짜 서버인 것처럼 요청을 다룬다. 그래서 클라이언트는 자신이 게이트웨이와 통신하고 있다는 사실을 알아채지 못한다.

예를 들어 HTTP/FTP 게이트웨이는 HTTP 요청을 받아 FTP 서버에서 문서를 가져온 뒤, 그 문서를 HTTP 메시지에 담아 클라이언트에게 돌려준다.

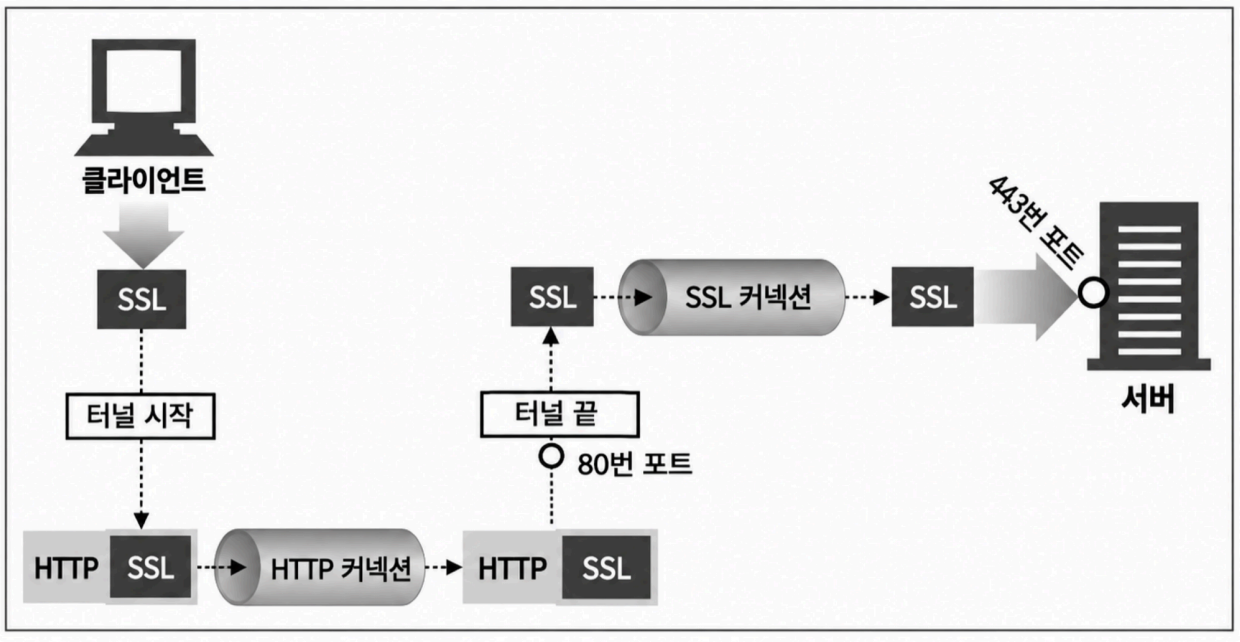


> 프록시와 게이트웨이의 차이는 **프로토콜 변환** 여부다. 프록시는 같은 프로토콜(HTTP)끼리 중개하고, 게이트웨이는 서로 다른 프로토콜을 연결한다.

1.7.4 터널

터널은 두 커넥션 사이에서 **raw 데이터를 열어보지 않고 그대로 전달**해주는 HTTP 애플리케이션이다. 주로 비 HTTP 데이터를 하나 이상의 HTTP 연결을 통해 그대로 전송하기 위해 사용된다.

> 대표적인 예가 암호화된 SSL 트래픽을 HTTP 커넥션으로 전송해서, 웹 트래픽만 허용하는 사내 방화벽을 통과시키는 것이다.



HTTP/SSL 터널은 HTTP 요청을 받아들여 목적지의 주소와 포트 번호로 커넥션을 맺는다. 이후로는 암호화된 SSL 트래픽을 HTTP 채널을 통해 목적지 서버로 전송할 수 있게 된다. 이때 터널을 여는 요청에는 `CONNECT` 메소드를 사용한다.

```
CONNECT www.example.com:443 HTTP/1.1
Host: www.example.com:443
```

터널은 암호화된 내용을 해석할 수 없고, 해석할 필요도 없다. 바이트를 양쪽으로 옮겨주기만 한다. (현재는 SSL의 후속인 **TLS**가 쓰이지만, 관용적으로 SSL이라는 이름도 함께 쓴다)

1.7.5 에이전트

사용자 에이전트는 사용자를 위해 HTTP 요청을 만들어주는 클라이언트 프로그램이다. 웹 요청을 만드는 애플리케이션은 무엇이든 HTTP 에이전트다. 지금까지 이야기한 웹 브라우저도 사용자 에이전트의 한 종류다.

그 외에도 사용자 에이전트는 여러 종류가 있다. 예를 들어 사람의 통제 없이 스스로 웹을 돌아다니며 HTTP 트랜잭션을 일으키고 콘텐츠를 받아오는 자동화된 사용자 에이전트가 있다. 검색 엔진이 웹 페이지를 수집할 때 쓰는 **크롤러(스파이더)** 가 대표적이다.

에이전트는 요청을 보낼 때 `User-Agent` 헤더에 자신이 누구인지 적어 보낸다. 서버는 이 값을 보고 모바일 브라우저인지, 검색 엔진 봇인지 구분할 수 있다.